

Manual Técnico

Hotel Leto — Documentación Técnica del Sistema

Este manual describe la arquitectura, estructura del código, modelos de datos, patrones de diseño y decisiones técnicas tomadas en el desarrollo de la aplicación web Hotel Leto. Está dirigido a desarrolladores que necesiten mantener, extender o auditar el sistema.

Proyecto	Hotel Leto — TFG Desarrollo de Aplicaciones Web
Alumno	Darío
Tecnologías	React · TypeScript · Vite · Supabase · Stripe · EmailJS
Despliegue	Vercel (Frontend) · Supabase Cloud (Backend/BD)
Fecha	Mayo 2025

Índice de Contenidos

1. Visión general del proyecto
2. Stack tecnológico
3. Estructura del proyecto
4. Arquitectura de la aplicación
 - 4.1 Flujo de datos
 - 4.2 Sistema de autenticación y roles
5. Modelo de datos (Supabase / PostgreSQL)
 - 5.1 Tablas principales
 - 5.2 Funciones RPC
6. Componentes y vistas
 - 6.1 Vistas principales
 - 6.2 Componentes reutilizables
 - 6.3 Custom Hooks
7. Integración con servicios externos
 - 7.1 Supabase Auth
 - 7.2 Stripe — Pagos
 - 7.3 EmailJS — Correos
 - 7.4 Anthropic Claude — IA
8. Enrutado
9. Estilos y diseño
10. Configuración de build y despliegue

1. Visión general del proyecto

Hotel Leto es una Single Page Application (SPA) desarrollada con React y TypeScript que permite la gestión online de reservas para un hotel boutique ubicado en el centro histórico de Mérida. El sistema implementa autenticación de usuarios, dos niveles de rol (cliente y administrador), reservas de habitaciones con pago mediante Stripe, reservas del salón de eventos, comunicaciones por email y un asistente virtual con IA.

2. Stack tecnológico

Tecnología	Versión	Rol en el proyecto
React	18.x	Librería de UI, gestión del árbol de componentes
TypeScript	5.x	Tipado estático, interfaces, mayor robustez
Vite	5.x	Bundler ultrarrápido, servidor de desarrollo HMR
Tailwind CSS	4.x (plugin Vite)	Estilos utility-first, variables CSS personalizadas
React Router DOM	6.x	Enrutado SPA declarativo
Supabase JS Client	2.x	SDK oficial para Supabase (BD, Auth, RPC)
Stripe.js / React Stripe	latest	Formulario de pago seguro (Elements)
EmailJS Browser	3.x	Envío de correos desde el frontend
Lucide React	latest	Iconografía SVG consistente
jsPDF	latest	Generación de PDFs en el cliente (pdfGenerator hook)

3. Estructura del proyecto

```
Hotel/  
  ■■■ src/  
    ■ ■■■ components/ # Componentes de UI reutilizables  
    ■ ■ ■■■ auth/ # AuthInput, AuthLayout  
    ■ ■ ■■■ landing/ # Secciones de la página principal  
    ■ ■ ■■■ layout/ # Header, Footer, navegación  
    ■ ■ ■■■ ui/ # Button, Typography, Map, CheckoutForm  
    ■ ■■■ hooks/ # Custom Hooks (lógica de negocio)  
    ■ ■■■ imgs/ # Recursos gráficos estáticos  
    ■ ■■■ lib/ # supabaseClient.ts (instancia singleton)  
    ■ ■■■ services/ # emailService.ts  
    ■ ■■■ styles/ # global.css (variables CSS, clases base)  
    ■ ■■■ views/ # Páginas completas (rutas)  
    ■ ■■■ App.tsx # Definición de rutas  
    ■ ■■■ main.tsx # Punto de entrada  
    ■■■ vercel.json # Rewrite SPA para Vercel  
    ■■■ vite.config.ts # Configuración Vite + plugins  
    ■■■ package.json
```

4. Arquitectura de la aplicación

4.1 Flujo de datos

La aplicación sigue el patrón de separación de responsabilidades mediante Custom Hooks. Las vistas (views/) son componentes de presentación que delegan toda la lógica a hooks personalizados (hooks/). Los hooks se comunican con Supabase a través del cliente singleton (lib/supabaseClient.ts).

- Vista (view) → llama al hook personalizado.
- Hook → gestiona estado local (useState/useEffect) y llama a Supabase.
- Supabase → devuelve datos de PostgreSQL vía PostgREST o funciones RPC.
- Hook → actualiza el estado y la vista se re-renderiza.

4.2 Sistema de autenticación y roles

La autenticación está gestionada íntegramente por Supabase Auth. Al iniciar sesión, se obtiene la sesión del usuario y se consulta la tabla 'profiles' para determinar su rol.

Existen dos guards de ruta implementados en useAuthGuard.ts:

- useRequireAuth(): redirige a /login si no hay sesión activa.
- useRequireAdmin(): redirige a / si el usuario no tiene rol 'admin'.

El Header comprueba el rol en cada renderizado para mostrar u ocultar el menú Admin dinámicamente.

5. Modelo de datos (Supabase / PostgreSQL)

5.1 Tablas principales

Tabla: profiles

Campo	Tipo	Descripción
id	UUID (FK auth.users)	Identificador del usuario (clave primaria)
nombre	TEXT	Nombre del usuario
apellidos	TEXT	Apellidos del usuario
telefono	TEXT	Teléfono de contacto (opcional)
rol	TEXT	'cliente' o 'admin'
created_at	TIMESTAMPTZ	Fecha de creación del perfil

Tabla: habitaciones

Campo	Tipo	Descripción
id	SERIAL (PK)	Identificador de la habitación
tipo	TEXT	Tipo de habitación (Suite Premium, Doble Estándar, Individual)
precio_por_noche	NUMERIC	Precio base por noche en euros
capacidad	INTEGER	Número máximo de huéspedes
descripcion	TEXT	Descripción de la habitación
disponible	BOOLEAN	Indica si la habitación está activa

Tabla: reservas

Campo	Tipo	Descripción
id	SERIAL (PK)	Identificador de la reserva
usuario_id	UUID (FK profiles)	Usuario que realizó la reserva
habitacion_id	INT (FK habitaciones)	Habitación reservada
fecha_entrada	DATE	Fecha de entrada
fecha_salida	DATE	Fecha de salida
precio_total	NUMERIC	Precio total pagado (habitación + servicios)
estado	TEXT	'pendiente', 'activa', 'pasada', 'cancelada'
created_at	TIMESTAMPTZ	Marca temporal de creación

Tabla: reservas_salon

Campo	Tipo	Descripción
id	SERIAL (PK)	Identificador
email_cliente	TEXT	Email del cliente para el evento
fecha_evento	DATE	Fecha del evento
precio	NUMERIC	Precio acordado
created_at	TIMESTAMPTZ	Fecha de creación

5.2 Funciones RPC

Función	Parámetros	Descripción
obtener_reservas_completas()	—	JOIN de reservas + profiles + habitaciones. Devuelve todas las reservas completas.
crear_reserva_salon()	p_email, p_fecha, p_precio	Inserta una nueva reserva de salón de eventos.
eliminar_mi_cuenta()	—	Elimina en cascada todos los datos del usuario autenticado. Protección de datos.

6. Componentes y vistas

6.1 Vistas principales

Vista	Ruta	Descripción
LandingPage	/	Página principal con todas las secciones del hotel
LoginPage	/login	Formulario de inicio de sesión
RegisterPage	/register	Formulario de registro de usuario
ForgotPasswordPage	/forgot-password	Solicitud de recuperación de contraseña
UpdatePasswordPage	/update-password	Formulario de nueva contraseña
VerificationPage	/verify	Pantalla tras verificar el email
ReservationPage	/reservation	Formulario de reserva de habitación + Stripe
AIChatPage	/ai-reservation	Chat con el asistente Leto AI
ReservationEventRoomPage	/reservation-event-room	Formulario reserva salón (cliente)
ReserveEventRoom (Admin)	/admin/reservation-event-room	Reserva salón desde admin
MyReservationsPage	/my-reservations	Historial de reservas del cliente
ProfilePage	/profile	Perfil del usuario
HistoryPage	/history-info	Historia y contexto arqueológico
ReservationManagementPage	/admin/reservation-management	Panel admin: gestión reservas
UserManagementPage	/admin/users	Panel admin: gestión de usuarios
RoomManagementPage	/admin/rooms	Panel admin: gestión de habitaciones
CreateUserPage	/admin/create-user	Panel admin: alta de cliente

6.2 Componentes reutilizables

Componente	Descripción
Header	Cabecera fija con logo, menús desplegables, toggle de tema y estado de sesión
Footer	Pie con enlaces, contacto y acceso empleados
AuthLayout	Layout compartido para todas las pantallas de autenticación
AuthInput	Input con icono, label y toggle de visibilidad para contraseñas
Button	Botón con variantes: primary, secondary, info, danger, ghost
CheckoutForm	Formulario de Stripe Elements para introducir datos de tarjeta
Map	Mapa de ubicación del hotel
Typography	Componentes Heading y Text con variantes de estilo

6.3 Custom Hooks

Hook	Responsabilidad
useLogin	Lógica de inicio de sesión con Supabase Auth
useRegister	Lógica de registro: validaciones + creación de cuenta + perfil
useAuthGuards (useRequireAuth / useRequireAdmin)	Protección de rutas según sesión y rol
useReservation	Estado del formulario de reserva, cálculo de precio, llamada a Stripe
useReservationManagement	Carga, filtrado y gestión de todas las reservas (admin)
useReservationEventRoom	Formulario y submit de reserva del salón
useRoomManagement	CRUD de habitaciones (admin)
useUserManagement	Carga y eliminación de usuarios (admin)
useCreateUser	Alta manual de cliente por parte del admin
useAIChat	Gestión del historial de mensajes y llamadas a la API de IA
userPasswordRecovery	Flujo de recuperación y actualización de contraseña
useFileDropZone	Gestión de arrastrar y soltar archivos
pdfGenerator	Generación de PDF de confirmación de reserva en el cliente

7. Integración con servicios externos

7.1 Supabase Auth

El cliente de Supabase se inicializa como singleton en `lib/supabaseClient.ts` con las variables `VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY`. Se usa `supabase.auth` para registro, login, logout, recuperación de contraseña y escucha de cambios de sesión (`onAuthStateChange`).

7.2 Stripe — Pagos

El flujo de pago tiene dos fases:

- Backend (Supabase Edge Function 'crear-pago-stripe'): recibe el precio total, crea un `PaymentIntent` en Stripe y devuelve el `clientSecret`.
- Frontend (`CheckoutForm` + `Stripe Elements`): usa el `clientSecret` para renderizar el formulario de tarjeta y confirmar el pago con `stripe.confirmPayment()`.

Si el pago falla, la reserva se elimina de la base de datos mediante `rollback` explícito en `useReservation`.

7.3 EmailJS — Correos

Se utiliza `emailjs-browser` para enviar correos directamente desde el frontend sin servidor propio. El servicio `emailService.ts` expone dos métodos:

- `enviarConfirmacionReserva()`: envía el resguardo de reserva al cliente.
- `enviarMensajeInformativo()`: usado por el admin para comunicados y por el sistema para confirmaciones del salón.

7.4 Anthropic Claude — IA

El asistente Leto AI (`AIChatPage` / `useAIChat`) realiza llamadas a la API de Anthropic Claude para responder consultas de los usuarios sobre el hotel, habitaciones, disponibilidad y servicios. El hook `useAIChat` gestiona el historial de mensajes y el indicador de 'escribiendo...'.

8. Enrutado

El enrutado está implementado con `React Router DOM v6`. `App.tsx` define todas las rutas de la aplicación. Las rutas de administración (`/admin/*`) aplican `useRequireAdmin()` para protección. Las rutas de usuario autenticado (`/reservation`, `/my-reservations`, `/profile`, `/ai-reservation`) aplican `useRequireAuth()`.

■ ■ El archivo `vercel.json` redirecciona todas las rutas a `index.html` para evitar 404 en navegación directa, ya que es una SPA sin servidor de rutas propio.

9. Estilos y diseño

Los estilos se gestionan con `Tailwind CSS v4` (plugin oficial de Vite) y un archivo `global.css` con:

- Variables CSS personalizadas (`--brand-rust`, `--brand-yellow`, `--text-main`, etc.) para modo claro y oscuro.

- Clases reutilizables definidas en @layer components: btn-primary, btn-secondary, input-primary, hero-wrapper, room-card, header, etc.
 - El modo oscuro se activa/desactiva añadiendo/eliminando la clase 'dark' en document.documentElement y se persiste en localStorage con la clave 'leto_theme'.
 - Fuentes: Inter (sans-serif) para cuerpo de texto y Antiqua Modern / Playfair Display (serif) para encabezados.
 - Paleta principal: Rust (#9c3f27), Yellow (#f0c34f), fondos claros/oscuros adaptativos.
-

10. Configuración de build y despliegue

Comando	Descripción
npm run dev	Servidor de desarrollo con HMR en localhost:5173
npm run build	Build de producción optimizado en dist/
npm run preview	Previsualización del build de producción
npm run lint	Análisis de código con ESLint (TypeScript + React)

Vite genera el build con code splitting, tree shaking y assets con hash para caché óptima. Vercel detecta automáticamente el framework Vite y ejecuta npm run build, sirviendo el contenido de dist/ desde su red CDN global con HTTPS automático.

El archivo tsconfig.app.json configura TypeScript en modo estricto (strict: true) con target ES2022, lo que garantiza la máxima detección de errores en tiempo de desarrollo.